

Strategic Warehouse Location Planner

Powered by AI

Warehouse Location Recommendation Agent

Solution, architecture, business value and submission document

Item	Detail
Team	Accenture
Document owner	Mayuresh Kolhatkar
Version	1.1 (verified against the deployed agents, 02 Oct 2026)
Date	02 October 2026
Review status	Draft. Technical and business review pending
Classification	Public. Hackathon submission
Submission deadline	02 October 2026, 11:59 PM PST

Contents

Business Problem and Target Users.....	3
Business problem solved.....	3
Who will use it.....	4
A strategic tool, not an operational one.....	4
Setup, Configuration, and Run Guide.....	4
Before You Start.....	4
Step 1: Set Up the Oracle Data Layer.....	5
Step 2: Set Up the MCP Server.....	6
Step 3: Set Up Azure Maps.....	6
Step 4: Configure the Geo Agent.....	6
Step 5: Configure the Recommendation Agent.....	7
Step 6: Smoke Test Before Running.....	8
Step 7: Run Sequence.....	8
Expected Output.....	9
Switching Operating Unit.....	9
Quick Troubleshooting.....	9
Architecture and Approach.....	9
The Approach in One Line.....	9
Design Principles.....	9
Architecture Layers.....	10
The Oracle Data Layer.....	11
Two Specialised Agents.....	11
How the Ranking Works.....	12
Built-In Checks.....	12
Why This Approach.....	13
Architecture Diagram and End-to-End Flow.....	14
Flow (Numbers Match the Diagram).....	14
Test Evidence and Known Limitations.....	15
Test Evidence.....	15
Oracle AI Database@Azure, Oracle MCP and Microsoft IQ.....	18
Web IQ: what we tried.....	19
Why we did not force Fabric IQ and Foundry IQ into this build.....	19

Project Evolution and Key Design Decisions.....	20
Timeline.....	20
Decisions that shaped the design.....	21
Lessons learned.....	22
Business Value and Expected ROI.....	23
Why the decision matters.....	23
Expected benefits.....	23
How we would size ROI with a client.....	24
Production Readiness Assessment.....	25
Microsoft Marketplace Readiness Assessment.....	26
Bonus Innovation Highlights.....	27
Demo Script for Judges.....	28
Prompts used in the demo.....	28
Contribution summary and acknowledgements.....	30

Business Problem and Target Users

Business problem solved

- **Warehouse location is a capital decision, but it's usually made with partial information.** Demand history sits in Oracle E-Business Suite (EBS). Market conditions such as rent, labour, transport links and risk are spread across reports and websites.
- **Oracle EBS stores addresses, not map coordinates.** You can't measure how far customers are from current or proposed warehouses without outside geocoding.
- **EBS has no record of warehouses that don't exist yet.** Candidate sites usually come from someone's judgement.
- **Manual analysis is slow and hard to repeat.** Pulling data, geocoding, researching markets and scoring in spreadsheets can take weeks. It rarely gets redone when assumptions change.
- **The results are hard to defend.** Committees get conclusions without the working, so they can't easily check the scoring, the sources or the data gaps.

What the solution does: It turns Oracle EBS shipment history into a ranked, repeatable shortlist of US warehouse locations. Each shortlisted site gets market research with sources, and the output is an Excel workbook that states its own limitations.

Who will use it

- **Supply chain and network planners (main users):** ask for a recommendation, test different service levels, regions or weightings, and re-run when needed.
- **Capital committee and finance reviewers:** review the workbook, where the scoring, the research and the limitations are shown separately.
- **Supply chain and operations leaders:** decide which sites go on to detailed due diligence.
- **Oracle EBS and database administrators:** set up read-only access and control the two reference tables the agents are allowed to update.

A strategic tool, not an operational one

- **It answers:** "Where should our next warehouse go, given where demand actually is?"
- **It does not handle** daily warehouse work such as picking, slotting, order release or delivery scheduling.
- **It looks at months or years of shipment history**, not live transactions.
- **It supports decisions with long payback periods.** These include network expansion, consolidation and lease commitments.
- **It helps people decide; it doesn't decide for them.** The agent recommends a site and explains the trade-offs. The planner and the committee make the call.
- **Major commercial and regulatory factors require separate due diligence.** The detailed exclusions are recorded in the limitations section.

Setup, Configuration, and Run Guide

Before You Start

- **Oracle E-Business Suite database:** an instance you can query. We use the Vision demo data.
- **Database administrator (DBA) access:** someone who can create objects in a custom schema and grant privileges.
- **MCP server:** Oracle SQLcl's built-in MCP server (sql -mcp), connected to the EBS database as MCP_READER.
- **Microsoft Foundry project:** with Agent Service enabled and Claude Opus 4.8 deployed.
- **Azure Maps account:** in the same Azure subscription, or one you can reach from Foundry.
- **Files from the repository:**

- Geo Agent instructions
- Recommendation Agent instructions
- scoring.py
- build_report.py
- The database scripts (views, tables and grants)
- **Security:** don't put keys, passwords or connection strings in instruction files or the repository. Use placeholders such as <MCP_ENDPOINT> and <AZURE_MAPS_CONNECTION>.

Step 1: Set Up the Oracle Data Layer

- **Create five objects in schema MCP_VIEWS:**
 - XXWLA_DEMAND_V: shipped demand by operating unit, period, region and postal code (view)
 - XXWLA_NETWORK_V: existing warehouses and their addresses (view)
 - XXWLA_GEO_PENDING_V: postal codes that don't have coordinates yet (view)
 - XXWLA_GEO_POSTAL_CACHE: coordinates for each postal code, with source (table)
 - XXWLA_CANDIDATE_SITE: candidate and existing warehouse sites (table)
- **Grant read access** on all five objects to the MCP user.
- **Grant write access on the two tables only.** Leave out DELETE on the geocode table, so the database itself stops geocodes being removed:

```
GRANT INSERT, UPDATE ON MCP_VIEWS.XXWLA_GEO_POSTAL_CACHE TO MCP_READER
GRANT INSERT, UPDATE, DELETE ON MCP_VIEWS.XXWLA_CANDIDATE_SITE TO
MCP_READER
```

- **Check the grants** by running these through the MCP server, one at a time:

```
SELECT SYS_CONTEXT('USERENV','SESSION_USER') AS session_user FROM dual
SELECT table_name, privilege FROM all_tab_privs
WHERE table_schema = 'MCP_VIEWS' AND grantee = 'MCP_READER'
AND table_name LIKE 'XXWLA%' ORDER BY table_name
```

- **Expected result:** SELECT on all five objects, plus write privileges on the two tables only.

Step 2: Set Up the MCP Server

- **Run Oracle SQLcl's built-in MCP server (sql -mcp) on a VM in the private Azure VNet, connecting as MCP_READER.** mcp-proxy exposes it over SSE behind nginx, and a VNet-integrated Azure Function relays it to Foundry. The repository has the service unit, nginx configuration and a setup script.
- Confirm auto-commit behaviour with the MCP server owner. Each call may run in a new session, so a COMMIT sent in a separate call might do nothing.
- Note the endpoint for Foundry: <MCP_ENDPOINT>.

Step 3: Set Up Azure Maps

- Create an Azure Maps account, or reuse an existing one.
- **Grant the Foundry account's managed identity the Azure Maps Data Reader role.** The agents call Azure Maps through an OpenAPI tool with managed-identity authentication, so there is no key to store or rotate.
- **Add the azure_maps OpenAPI tool (tools/openapi/azure_maps.json in the repository) to both agents.** The portal only offers connection authentication for OpenAPI tools, so managed identity is set through the agent API; scripts/deploy_agent.py does this.
- **Test the tool on its own before relying on it:**
 - Use Azure Maps to return latitude and longitude for postal code 10001, New York, NY, US.
 - Show the raw tool response.
 - **Expected result:** roughly 40.75, -73.99, returned by the tool rather than from the model's memory.

Step 4: Configure the Geo Agent

- **Name:** Hackathon-Oracle-Geo-Postal-code-generator
- **Instructions:** paste the current Geo Agent instruction file.
- **Tools:**
 - **MCP server:** read and write. Set approval to **Never require approval**.
 - **Azure Maps:** the only source of coordinates the agent looks up (Modes E, R and B).
 - **Web search:** optional, only for the infrastructure note on candidate sites (Mode B).
- **Code Interpreter:** not used by the instructions. All calculations for this agent run in SQL.

- **Keep these tool-call rules in the instructions:**
 - One SQL statement per MCP call
 - No semicolon at the end of a statement
 - Use the approved SQL exactly as written; replace only the placeholders
 - Prefix every object with MCP_VIEWS.
 - If a call fails, report the exact Oracle error and stop

Step 5: Configure the Recommendation Agent

- **Name:** Hackathon-Oracle-Warehouse-Recommendation-Agent
- **Instructions:** paste the current Recommendation Agent instruction file.
- **Tools:**
 - **MCP server:** SELECT only, enforced by the read-only rule in the instructions. Set approval to **Never require approval**.
 - **Azure Maps:** checking the coordinates of shortlisted and candidate sites.
 - **Web search:** Tier 2 research on rent, labour, infrastructure and risk, for shortlisted sites only.
 - **Code Interpreter:** on.
- **Attach exactly two files to Code Interpreter:** scoring.py and build_report.py.
 - Remove any old CSV, TXT or test files.
 - Keep only one file per name. The scripts find files by name fragment, so an old copy could be picked up instead.
- **Keep the same tool-call rules** as the Geo Agent.
- **Recommended:** a separate read-only MCP connection for this agent. Today both agents share MCP_READER, which now has write access.

Step 6: Smoke Test Before Running

- **Geo Agent:**
 - Smoke test. Operating unit: Vision Operations. Period: 01-FEB-2002 to 01-OCT-2010.
 - Run the preflight check and report postal codes and total units.
 - Run the status query and show the raw result.
 - Use Azure Maps for ONE pending postal code, write it, and read it back.
 - Show what a Mode C reset would delete. Do NOT delete.
 - For each step show the exact SQL or tool call and the raw result. End with PASS/FAIL per step.

- **Recommendation Agent:**
- Smoke test. Operating unit: Vision Operations. Period: 01-FEB-2002 to 01-OCT-2010. Region: ALL.
 - Run Steps 1 and 2 only. Show the exact SQL and raw result for each.
 - List /mnt/data and report the row count of each file written.
 - Confirm the demand file row count equals postal_codes from Step 1a.
- **A pass means:** real numbers and the raw results are shown. If the agent says a value is "expected" or hands your instructions back to you, that's a fail.

Step 7: Run Sequence

- **1. Check status (Geo Agent):**
 - Status. Operating unit: Vision Operations. Period: 01-FEB-2002 to 01-OCT-2010.
- 2. Geocode demand (Geo Agent, Mode E with Azure Maps). Repeat until at least 80% of units are geocoded and no warehouse is missing coordinates:
 - Mode E. Operating unit: Vision Operations. Period: 01-FEB-2002 to 01-OCT-2010.
 - Geocode the next batch of pending postal codes using Azure Maps, then show status.
- **3. Generate candidates (Geo Agent, Mode B):**
 - Mode B. Operating unit: Vision Operations. Period: 01-FEB-2002 to 01-OCT-2010.
 - Region: ALL. Propose candidate markets for a new warehouse.
- **4. Get the recommendation (Recommendation Agent):**
 - Recommend a location for a new warehouse. Operating unit: Vision Operations.
 - Period: 01-FEB-2002 to 01-OCT-2010. Region: ALL. Use default weights.
- **5. Download the workbook straight away.** Code Interpreter files only last for the session.

Expected Output

- **Chat summary:**
 - Parameters used, and which ones were defaults
 - The recommended site and a one-line reason
 - Whether the pick differs from computed rank 1, and why

- The geocoded percentage, marked **PROVISIONAL** if below 80%
- **Excel workbook:** the calculated ranking, the research with sources, the recommendation and the limitations, each on its own sheet.

Switching Operating Unit

- Run **Mode C** in the Geo Agent. It shows what would be deleted and asks you to confirm.
- The geocode table is kept. It only gains rows over time.

Quick Troubleshooting

- If the Recommendation Agent pauses during a high-load period, it retains the completed-step status. Prompt it with “**Start with the next incomplete step**” to resume processing.

Architecture and Approach

The Approach in One Line

The code does the maths, the AI does the judgement, and Oracle stays the system of record.

Design Principles

- **Calculations come from code, not the AI.** Distances, coverage and scores are calculated by a fixed script, so the same question always gives the same ranking.
- **The AI handles what code can't.** It weighs market evidence that is uneven or contradictory, and explains the trade-offs in plain business language.
- **Calculated and researched results stay separate.** The demand-fit score and the market findings are never blended into one number. Blending them would create false precision.
- **Every limitation is stated.** Each report shows how complete the data was and lists what wasn't evaluated.
- **Guardrails are set in more than one place.** Key rules are written into the agent instructions, checked by the scripts and, where it matters most, enforced by Oracle permissions.

Architecture Layers

Layer	Components	Role
Data	Oracle E-Business Suite on Oracle	System of record for demand and

	Exadata Database Service on Oracle Database@Azure; custom schema MCP_VIEWS	the warehouse network. Also stores the location data and candidate sites the agents add
Access	Oracle SQLcl MCP server on a private VNet VM, reached through a VNet-integrated Azure Function relay	The only route from the agents into Oracle. What the agents can do is controlled by Oracle grants
Intelligence	Microsoft Foundry Agent Service, Claude Opus 4.8, and two specialised agents	Lookups, research, reasoning and running the scripts
Location	Azure Maps	Accurate, repeatable map coordinates for postal codes and sites
External evidence	Foundry web search (Microsoft Web IQ was planned; see the Microsoft IQ section)	Market research for shortlisted sites, with sources
Computation	Code Interpreter running scoring.py and build_report.py	Ranking calculation and a checked Excel report
Output	Excel workbook and a short chat summary	A decision pack a committee can review

The Oracle Data Layer

- **Read-only views over standard EBS tables:**
 - XXWLA_DEMAND_V: shipped demand by operating unit, month, region and postal code. Measures are units, weight, volume and amount.
 - XXWLA_NETWORK_V: existing warehouses and their addresses.
 - XXWLA_GEO_PENDING_V: postal codes that don't have coordinates yet.
- **Two tables the agents maintain:**
 - XXWLA_GEO_POSTAL_CACHE: coordinates for each postal code, with resolution level and source. Geocoding is done once and reused.
 - XXWLA_CANDIDATE_SITE: proposed sites, plus the existing warehouses as a comparison baseline.
- **Least-privilege access:** read access applies to all five objects, writes are restricted to the two controlled tables, and geocode deletion is blocked. Exact grants are documented in the setup guide.

Two Specialised Agents

Geo Agent: Prepares the Data

- **Mode E (geocoding):** looks up coordinates for pending postal codes with Azure Maps, starting with the codes that carry the most volume. Results are written back to Oracle and read back to confirm the write.
- **Mode B (candidate sites):** proposes US markets around the demand-weighted centre, uses Azure Maps for their coordinates, and stores them as candidates.
- **Mode C (reset):** clears the candidates when the operating unit changes. It asks for confirmation first.
- **Mode R (reverse lookup):** returns the address for a latitude and longitude. Writes nothing.
- **Modes M and BM (manual entry):** store geocodes or candidate sites supplied by the user, for the cases Azure Maps cannot resolve.
- **Status:** reports how ready the data is, for example the percentage of units geocoded.
- **It never** ranks or recommends sites.

Recommendation Agent: Decides and Explains

- **Tier 1, Compute:** pulls data from Oracle with read-only queries. `scoring.py` checks the totals against the database before ranking anything, then ranks the candidates.
- **Tier 2, Research:** looks up rent, labour, infrastructure and risk for the shortlist only. Every finding is cited and classified as favourable, adverse, neutral or absent.
- **Tier 3, Reason:** recommends one site and explains the trade-off. If it picks anything other than the top-ranked site, it must say why.
- **Report:** `build_report.py` builds the workbook and refuses to build it if the research breaks the rules.
- **It never** changes Oracle data or calculates a number itself.

How the Ranking Works

- **Proximity:** how far each candidate is from demand, weighted by volume.
- **Coverage:** the share of demand reachable within the service-time target (default 8 hours).
- **Normalisation and weighting:** both measures are put on the same scale and combined. Default weights are 60% proximity and 40% coverage; users can change them.
- **Overlap penalty:** a candidate close to an existing warehouse loses up to 25% of its score.

- **Confidence:** if less than 80% of demand units are geocoded, the result is marked **PROVISIONAL**.

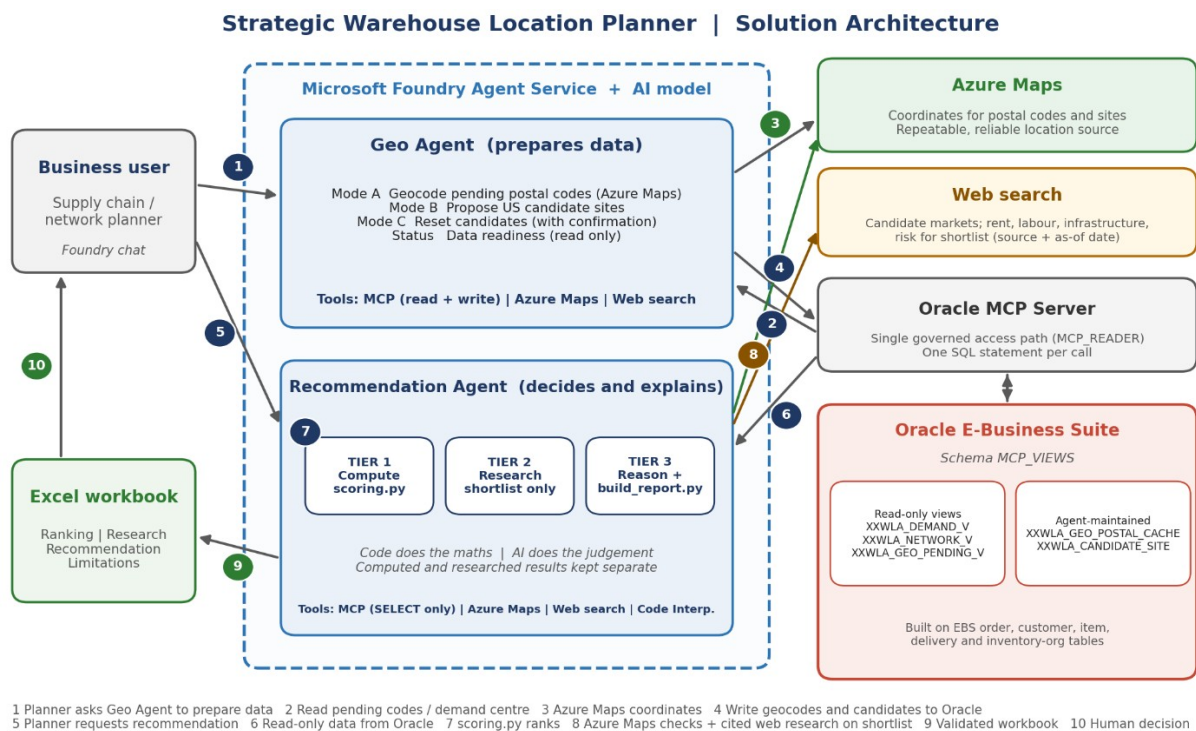
Built-In Checks

Risk	Control
The AI invents or changes numbers	Scores come only from scoring.py; the report script copies them directly
Rows lost between Oracle and scoring	Totals from the database are checked before scoring starts
Bad or made-up coordinates	Coordinates come from Azure Maps; Oracle rejects 0,0 or unsourced entries; a better result is never overwritten by a worse one
Unintended data changes	Write access on two tables only, no delete on geocodes, and a read-only rule for the Recommendation Agent
Research drifting off the shortlist	The report script refuses to build if research goes outside the shortlist
Poorly documented markets penalised unfairly	"No data found" is kept separate from "unfavourable"
Over-reliance on the output	Every report lists its limitations; people make the final decision

Why This Approach

- **It uses data organisations already have.** EBS shipment history becomes a basis for network strategy.
- **It is repeatable and auditable.** The same inputs give the same ranking, and every finding has a source.
- **It improves over time.** Geocodes and candidates stay in Oracle, so each run starts from a better position.
- **It can grow.** Natural next steps:
 - Move the scoring into Oracle or Microsoft Fabric for larger volumes.
 - Have the Recommendation Agent call the Geo Agent directly as a sub-agent.
 - Add structured cost and labour data.

Architecture Diagram and End-to-End Flow



Flow (Numbers Match the Diagram)

#	Step
1	Planner asks the Geo Agent to prepare data
2	Geo Agent reads pending postal codes and the demand centre through MCP
3	Azure Maps returns coordinates
4	Geo Agent writes geocodes and candidates to Oracle
5	Planner asks the Recommendation Agent for a recommendation
6	Agent reads demand, network, geocodes and candidates (read only)
7	scoring.py ranks the candidates
8	Agent checks the shortlist's coordinates in Azure Maps and researches each site on the web, with citations
9	build_report.py checks the research and produces the workbook
10	Planner reviews the workbook and decides

Test Evidence and Known Limitations

Test Evidence

The agent generated the following test workbook:

- **File:** Warehouse_Recommendation_Vision_Operations_20261002_1349.xlsx (in submission/evidence/ in the repository)
- **Operating unit:** Vision Operations
- **Analysis period:** 01-FEB-2002 to 01-OCT-2010
- **Country:** US
- **Region:** ALL

Results of this run:

- **Ranking status:** FINAL. No overrides applied.
- **Oracle control totals:** 51 postal codes and 4,796,338 shipped units read through Oracle MCP; 99.9% of units geocoded.
- **Candidates:** 100 sites ranked by scoring.py; 82 existing inventory organisations used in the overlap test.
- **Recommendation:** Columbus OH, computed rank 1 (composite score 0.9894). It does not differ from rank 1.
- **Azure Maps location check:** all three shortlisted sites MATCH their recorded state (OH, VA, PA).
- **Market research:** 12 findings across the three shortlisted sites. 11 have a source and date; one (industrial rent, Oakton VA) is ABSENT because no valid source was found.
- **Coverage:** about 34% of demand is within the 8-hour service target of the recommended site, which the workbook states as a limitation.

Rank	Site	Score	Coverage %	Weighted km	Rent	Infrastructure	Labour	Risk
1	Columbus OH	0.9894	33.6	1,351	Neutral	Favourable	Neutral	Adverse
2	SSC US 02 (Oakton VA)	0.9145	33.7	1,641	Absent	Favourable	Neutral	Neutral
3	Harrisburg PA	0.9105	34.1	1,669	Neutral	Favourable	Neutral	Adverse

Limitations

Scope

Limitation	Impact	Mitigation / next step
One operating unit per analysis	No view across several business units	Run each unit separately; combine across units later
Recommends a single site	Doesn't optimise a network of several sites at once	Network optimisation is on the roadmap
Only sites in the candidate table can be ranked	A better location that was never proposed can't come up	Mode B proposes markets; the business can add or edit candidates. This limitation is stated in every report
Strategic decision support only	Doesn't make the final decision or handle day-to-day warehouse work	People keep the final decision

Ranking method

Limitation	Impact	Mitigation / next step
Straight-line distance × 1.25 road factor at 45 km/h	Not real drive time; distance is understated where road networks are poor	Use routed drive times from Azure Maps
Fixed assumptions:	Results depend on these settings	Users can change the service target and weights; all assumptions are printed in the output
Not evaluated: cost, tax,	Not enough on its own for final investment approval	Stated in every report; add structured cost and labour data
Up to 500 postal codes per run	Larger runs stop and ask the user to narrow the scope	Move scoring into Oracle or Fabric

Oracle AI Database@Azure, Oracle MCP and Microsoft IQ

The hackathon names six technologies. We would rather be straight about which ones this solution uses than draw them all into a diagram.

Technology	Role in this solution	Status	Notes
Oracle AI Database@Azure	Hosts Oracle E-Business Suite on Oracle Exadata Database Service; schema MCP_VIEWS holds the agents' views and tables	Built & tested	Vision demo data, analysis period 2002-2010
Oracle MCP	Oracle SQLcl's built-in MCP server; the only path from the agents into Oracle, running as MCP_READER	Built & tested	Private VNet, no public IP; Foundry reaches it through a VNet-integrated Azure Function
Microsoft Foundry Agent Service	Hosts both agents (Claude Opus 4.8) with MCP, Code Interpreter, OpenAPI and web search tools	Built & tested	Agents are versioned; definitions are kept as code in the repository
Azure Maps	Geocoding (Geo Agent) and the location check (Recommendation Agent)	Built & tested	Keyless: OpenAPI tool with the Foundry account's managed identity
Web IQ	Intended replacement for web search in Tier 2 market research	Attempted, not enabled	Web IQ rejected the Foundry managed identity (limited access). Foundry web search used instead
Fabric IQ	Not used in this build	Not used	Our companion Order-to-Ship Recovery Agent uses Fabric with mirrored EBS data. Future: mirror the demand view into Fabric for larger volumes
Foundry IQ	Not used in this build	Not used	Future: a knowledge base of approved market reports alongside web research
Work IQ	Not used in this build	Not used	Future: planner context from email and Teams, such as sites the business has

			already ruled out
--	--	--	-------------------

Web IQ: what we tried

We wanted Tier 2 market research to run on Microsoft Web IQ instead of the generic web search tool, because it returns structured results with source URLs that suit cited research. We configured Web IQ as an OpenAPI tool in Foundry using the Foundry account's managed identity, the same keyless pattern we use for Azure Maps. Web IQ accepted the token but rejected the identity: "Application ... is not authorized to access this service". Web IQ is currently limited access, and our tenant is not enrolled, so we could not add it before the deadline and kept Foundry web search. The tool definition is ready in the repository (tools/webiq/) and can be switched on once access is granted.

Why we did not force Fabric IQ and Foundry IQ into this build

This agent needs a few hundred aggregated rows per question, and the critical requirement was that every number be traceable to an Oracle query. A direct SQL path through Oracle MCP met that with the fewest parts. On our Order-to-Ship agent we have already seen what a semantic layer adds and what it costs in latency, so we would bring it in when volume or reuse justifies it, not before.

Project Evolution and Key Design Decisions

The agent did not arrive in its current shape on day one. It went through three distinct versions in about two weeks, and most of the changes were forced by real constraints: access rights, platform behaviour and data quality.

Timeline

When	What happened	What it changed
Before 15 Sep	Initial ask: an agent that recommends warehouse locations anywhere in the world, sourcing data from an MCP server. Parameters defined, then an EBS data requirements workbook mapping each need to EBS tables.	Too broad for a hackathon. Broken into agile phases, then a one-week MVP.
Design review	Scoring scripts on the MCP server did not work: ours only connects to the EBS database. Manual geocoding of the top locations would not scale.	A persistent geocode cache filled by a separate agent, and the three-tier model: code scoring, agent research, agent reasoning.
15-21 Sep	Hackathon Learn & Earn period, including the Oracle MCP session on 17 Sep. [TEAM INPUT REQUIRED]	
27 Sep	Creating tables in EBS needed DBA help, so a file-based version came first. CSV uploads were refused, so .txt files were used.	First end-to-end recommendation, flagged PROVISIONAL at 69.7% geocoded.
28-30 Sep	Back to Oracle as the source: parameters for operating unit, date range and region; the five MCP_VIEWS objects created; V2 agents built; multi-statement and privilege errors fixed; write grants applied.	Current data architecture.
01 Oct	Azure Maps replaced web search as the coordinate source, called through a keyless OpenAPI tool.	Repeatable coordinates; no keys in the solution.
02 Oct	End-to-end run on Vision Operations 2002-2010: FINAL ranking at 99.9% geocoded. Web IQ attempted for research and blocked by access. Solution packaged as code.	The test evidence in this document.

Decisions that shaped the design

Decision	What we chose	Why	Trade-off we accepted
Who does the maths	Code, never the model	A score the agent computed cannot be reproduced or audited	More moving parts (script, files, integrity checks)
Computed vs researched	Shown separately, never blended	Blending a measured distance with a researched rent estimate creates false precision	The reader does some of the weighing; the agent explains it
One agent or two	Two	Geocoding is a lookup with strict rules; recommending is judgement	The user talks to two agents; a sub-agent call is on the roadmap
Geocode at query time or cache	Persistent cache in Oracle	Postal codes do not move; runtime lookups are slow and not reproducible	Needs write access, which took time to get
Scoring in PL/SQL or Foundry	scoring.py in Code Interpreter for now	No rights to compile a package in MCP_VIEWS	Results reach the agent as text; mitigated by aggregation, a 500-postal-code cap and control totals
Geocode table permissions	No DELETE grant at all	The database itself enforces that geocodes are never deleted	Corrections are made by update only
Who builds the workbook	build_report.py, not the agent	Retyping is where numbers change	One more file to maintain
Azure Maps authentication	Managed identity, no key	Nothing to store, leak or rotate	Has to be set through the agent API; the portal offers only connection auth

Lessons learned

- **Do not trust an agent's explanation of its own failure.** The Geo Agent claimed the tool "does not allow multiple SQL runs". It had sent several statements in one call. One statement per call, no trailing semicolon and the exact Oracle error fixed it.
- **Agents report what should have happened.** A smoke test said values were "expected". We now require the exact SQL and the raw result in every test.

- **Check the privileges, not the error message.** ORA-01031 on writes came from MCP_READER having only SELECT. Three read-only queries diagnosed it in minutes.
- **Demo data has its own calendar.** The Vision transactions sit roughly in 2002-2010; a 2023-2026 window returned two postal codes.
- **Data readiness drives confidence.** At 69.7% geocoded the first ranking was correctly PROVISIONAL; after the Azure Maps backfill it reached 99.9% and FINAL.
- **Managed identity for Foundry tools comes from the Foundry account, not the project or the agent.** Grant roles to that identity.
- **Access is the critical path.** Every version change traced back to who could create, compile or write what in Oracle, and Web IQ access was the one thing we could not get in time.

Business Value and Expected ROI

Nothing below has been measured against a baseline, because the agent has not yet supported a live business decision. What we can show is why the decision is worth getting right, where the agent saves effort, and how we would measure it.

Why the decision matters

Our team's manual research workbook (16 Sep) costed a 100,000 sq ft building across US markets. All-in annual occupancy cost ranged from about \$737,500 in Indianapolis to about \$1,150,000 in Chicago among the first four markets, a gap of roughly \$412,500 a year before labour or freight. These figures come from public broker benchmarks and are illustrative, not agent output. Picking the wrong market is a six-figure annual mistake that compounds over a lease term.

Expected benefits

Benefit	Today	What the agent changes	KPI to track	Status
Faster first answer	Weeks of extract, geocode, research and spreadsheet work	One conversation once the geocode cache is warm	Time from question to reviewed shortlist	Expected
Repeatable comparison	Rebuilt by hand; hard to replay	Same inputs give the same ranking	Re-runs that reproduce the prior ranking	By design
Traceability for the committee	Conclusions without the working	Computed, researched and limitations on separate sheets; every finding cited	Review questions the workbook cannot answer	Expected
Less manual geocoding	Analyst time per decision	Cache fills once and is reused	Analyst hours per study	Expected
Better use of EBS data	Shipment history rarely used for network design	Demand by postal code feeds the ranking	Share of shipped units included	Measured per run (99.9%)

How we would size ROI with a client

Annual value = studies per year x analyst hours saved per study x loaded hourly rate, plus the avoided cost of a poorer site choice agreed with finance. Cost = Foundry and model consumption + MCP hosting + support effort. [TEAM INPUT REQUIRED]: number of siting studies a typical client runs per year and current hours per study.

Production Readiness Assessment

This is a working prototype with sound controls in its design, not a production system.

Area	In place and tested	Gap / next step
Security and data privacy	Least-privilege Oracle grants verified; agents limited to approved SQL; no DELETE on geocodes; MCP backend in a private VNet with no public IP; no keys anywhere (Azure Maps via managed identity); repository secret-scanned	Both agents share one MCP user; the MCP relay endpoints are anonymous because Foundry's MCP client drops function keys; data-handling review for web search queries
AI safety and evaluation	Refusal rules; scoring integrity check; report validation; mandatory limitations; ABSENT kept separate from ADVERSE; end-to-end run checked by hand	Golden set of test questions and automated evaluation not built; no prompt-injection testing on web content
Scalability and reliability	SQL aggregation; 500-postal-code cap; control totals	Single MCP VM; not load tested; move Tier 1 scoring into Oracle or Fabric for volume
Monitoring	Foundry run traces; Application Insights on the MCP relay	No dashboards, alerting or usage logging
Deployment and rollback	Agents versioned in Foundry; definitions, scripts, DDL and MCP configuration kept as code with export, dry-run diff and deploy scripts	No CI/CD pipeline yet

Microsoft Marketplace Readiness Assessment

A Marketplace offer is not part of this submission. [TEAM INPUT REQUIRED]: confirm whether Marketplace publication is planned and, if so, the intended offer type.

Topic	Where we are	What would be needed
Packaging	Agent definitions, scripts, DDL and MCP configuration in a repository with deploy scripts	One-step installer (for example Bicep or azd) that creates the schema objects, grants and both agents from parameters
Customer onboarding	Manual; needs EBS, Azure and Foundry skills	Guided setup; a file-upload mode for customers without DBA support
Multi-ERP / multi-country	Oracle EBS, US only	Country and region parameters, currency handling, other ERP views
Support model	Project team	Defined support tiers and contacts
Publishing requirements	Not started	Partner Center registration, offer listing, security and privacy documentation, certification

Bonus Innovation Highlights

Highlight	Why it is different	Status
AI write-back into Oracle with database-enforced limits	The Geo Agent enriches EBS with coordinates and candidate sites. Oracle grants, not just prompts, decide what it can change	Built & tested
Trust-but-verify pipeline	Control totals from SQL, an integrity check in the scorer, and a report builder that refuses bad research	Built & tested
Honest research classification	ABSENT vs ADVERSE stops the agent penalising under-documented locations	Built & tested
Keyless Azure Maps from Foundry	OpenAPI tool with managed identity, so no map key exists anywhere in the solution	Built & tested
Proposed new Oracle-Microsoft connection	Private Oracle MCP on Oracle Database@Azure reached from Foundry through a VNet-integrated relay; the geocode cache becomes a shared Oracle reference asset for other agents	Built (relay); future (shared cache)
Web IQ for cited research	Tool definition prepared; blocked by Web IQ access for the Foundry identity	Prepared, not enabled
Review of existing AI Blueprints	[TEAM INPUT REQUIRED]	

Demo Script for Judges

Target length is about four minutes. Record from a clean conversation and keep the Foundry run trace visible during the Oracle part, because judges look for evidence that Oracle data is retrieved and used.

Time	Segment	What to show
0:00-0:25	The problem	Siting a warehouse is a capital decision made from scattered data
0:25-0:50	The solution	Architecture diagram: two agents, three tiers, maths in code, judgement in the agent
0:50-1:40	Oracle evidence	Geo Agent status prompt; expand the trace to show the SQL sent to MCP_VIEWS and the raw result; list the candidate sites
1:40-2:50	Recommendation	Recommendation Agent prompt for Vision Operations 2002-2010: preflight control totals, Azure Maps location check, then the summary
2:50-3:30	The workbook	Open the Excel file and walk through the ranking, research, recommendation and limitations sheets
3:30-3:50	Guardrail moment	Ask "average the top three scores" or for a site not on the list, and show the refusal
3:50-4:10	Value and next steps	Faster, traceable first answer from EBS data; next: scoring in Oracle, evaluation set, Web IQ

Prompts used in the demo

- **Geo Agent, status:** Status only. Operating unit: Vision Operations. Period: 01-FEB-2002 to 01-OCT-2010. Run the status query and show the exact SQL and the raw result. Report total units, geocoded units %, pending postal codes and warehouses missing coordinates. Do not write, update or delete anything.
- **Geo Agent, candidate sites:** Read only. Operating unit: Vision Operations. List all active candidate sites and existing warehouse nodes in
XXWLA_CANDIDATE_SITE: site_id, site_name, region, city, state, latitude, longitude, is_existing_node. Show the exact SQL and the raw result. Do not write, update or delete anything.
- **Recommendation Agent:** Recommend a location for a new warehouse. Operating unit: Vision Operations. Period: 01-FEB-2002 to 01-OCT-2010. Region: ALL. Use default metric, service hours and weights, and state which defaults you applied. Show the preflight control totals from Oracle. Use Azure Maps to confirm the coordinates of

the shortlisted sites, and web search for market research with sources. Build the Excel workbook with `build_report.py` and give me the download link.

Contribution summary and acknowledgements

Designed and developed by the Accenture team. [TEAM INPUT REQUIRED]: team members and roles.